QSG-RM

Prerequisites

1. **Nash equilibrium and policy update given Qs**
   Given an agent POV, a nash equilibrium function requires the Q function of the agent and the other player. This returns the **policies** which are coherent with the nash equilibrium definition. So, the policies are updated so that **no player ends up requiring additional changes in the policy itself** if the other one has that fixed policy.
   The policies reaches a **nash equilibrium** in the sense that the agent are satisfied with their strategies, if the other player **keeps its policy as it is.**

   > Both policies are updated, and they stop when that contition is satisfied.

2. **Action Choices**
   Given an agent, actions are choosen accordingly to the epsilon-greedy policy: maximize its Q with probability epsilon.
3. **Off-Policy Method**
   The method is **off-policy**, since the finite episode has
   - Behavioural policy (the one the agent uses to move) → epsilon greedy (2.)
   - Target policy → nash equilibrium strategy (we assume there on players will play accordingly to Nash Equilibrium

Algorithm definition

1. **Initialization**
   We do initialize the game state and the RM states for each of the two agent specific RM. Since then each agent has to internally keep the q-function estimate for both its own POV and the opposite player POV, we need to initialize 4 Q functions too: 2 for player Ego, 2 for player Adv. Their initialization is not constraint at an algorithm level, but might be at the Nash Solver level.
2. **For each of the episodes…**
   a. **For each step of the current episode…**
      i. We have to perform an action
         1. For each agent (ego and adv) we have to **select which action to do in this step**. In order to perform this, we need to have the **current behavioral policy of the agent**. Since we've always defined that the agents are acting following the NE, strategies are **obtained by finding the current step nash equilibrium solution**: we find the policies that, given the current estimate of the Q function for both ego and adv are s.t. no one requires additional changes, under the assumption of the other player no-change-in-policy.
            a. So, we use the current Qs to get the current strategy. We use this to build the actual behavioral policy including the epsilon greediness.
            b. If so, we end up having the `current state action to perfom` by **each agent**.
         2. Each agent **execute its action simultaneusly**; they go to a **new coords** and we get a `new game state`
         3. Since we need to have an **immediate response,** we translate the tuple `start game state, actions performed, reached game state` into the Reward Machine `label` for this step.
            a. So, get the `new (internal) reward machine state` for both Ego and Adv POV (wrt to their own RM)
      ii. We have to update the Q-function current estimates
         1. At this moment we have `starting game state, actions performed, reached game states, initial and reached RM states` . We need to update the q-functions considering this states we'vere reached in both game and RM. The q function gets updated by having a component alpha dependent which is the previous q-function, i.e. what we knew up to that moment. Then, we have to add a 1-alpha weighted term, which is a measure of what we have learned thanks to that step we've done.
         2. So, **for each agent:**
            a. Its original q-functions where $q_{e,i}, q_{a,i}$ i.e. one for the Ego agent from its point of view and one for the Adv agent from its point of view. These are the one weighted by alpha.
            b. By the Bellman equations definition, we always have to consider the contribution of the following state in discounted cumulative reward, which is something **dependent on how the agent performs acting following Nash Equilibrium from there on**.

$$q_{\mathfrak{i}}^{*}(s, v_{\mathfrak{e}}, v_{\mathfrak{a}}, a_{\mathfrak{e}}, a_{\mathfrak{a}}) = r_{\mathfrak{i}}(s, v_{\mathfrak{e}}, v_{\mathfrak{a}}, a_{\mathfrak{e}}, a_{\mathfrak{a}}) + \gamma \sum_{\substack{s' \in S \\ v'_{\mathfrak{e}} \in V_{\mathfrak{e}} \\ v'_{\mathfrak{a}} \in V_{\mathfrak{a}}}} p(s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}} | s, v_{\mathfrak{e}}, v_{\mathfrak{a}}, a_{\mathfrak{e}}, a_{\mathfrak{a}}) \tilde{v}_{\mathfrak{i}}(s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}}, \pi_{\mathfrak{e}}^{*}, \pi_{\mathfrak{a}}^{*})$$

for $\mathfrak{i} \in \{\mathfrak{e}, \mathfrak{a}\}$ **do**

$\pi_{\mathfrak{e}\mathfrak{i}}(\cdot|s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}}), \pi_{\mathfrak{a}\mathfrak{i}}(\cdot|s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}}) \leftarrow \text{CalculateNashEquilibrium}(q_{\mathfrak{e}\mathfrak{i}}(s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}}), q_{\mathfrak{a}\mathfrak{i}}(s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}}))$

$\overline{q}_{\mathfrak{e}\mathfrak{i}}(s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}}) \leftarrow \pi_{\mathfrak{e}\mathfrak{i}}(\cdot|s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}})\pi_{\mathfrak{a}\mathfrak{i}}(\cdot|s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}})q_{\mathfrak{e}\mathfrak{i}}(s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}})$

$\overline{q}_{\mathfrak{a}\mathfrak{i}}(s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}}) \leftarrow \pi_{\mathfrak{e}\mathfrak{i}}(\cdot|s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}})\pi_{\mathfrak{a}\mathfrak{i}}(\cdot|s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}})q_{\mathfrak{a}\mathfrak{i}}(s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}})$

$q_{\mathfrak{e}\mathfrak{i}}(s, v_{\mathfrak{e}}, v_{\mathfrak{a}}, a_{\mathfrak{e}}, a_{\mathfrak{a}}) \leftarrow (1 - \alpha_t) q_{\mathfrak{e}\mathfrak{i}}(s, v_{\mathfrak{e}}, v_{\mathfrak{a}}, a_{\mathfrak{e}}, a_{\mathfrak{a}}) + \alpha_t (r_{\mathfrak{e},t} + \gamma \overline{q}_{\mathfrak{e}\mathfrak{i}}(s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}}))$

$q_{\mathfrak{a}\mathfrak{i}}(s, v_{\mathfrak{e}}, v_{\mathfrak{a}}, a_{\mathfrak{e}}, a_{\mathfrak{a}}) \leftarrow (1 - \alpha_t) q_{\mathfrak{a}\mathfrak{i}}(s, v_{\mathfrak{e}}, v_{\mathfrak{a}}, a_{\mathfrak{e}}, a_{\mathfrak{a}}) + \alpha_t (r_{\mathfrak{a},t} + \gamma \overline{q}_{\mathfrak{a}\mathfrak{i}}(s', v'_{\mathfrak{e}}, v'_{\mathfrak{a}}))$

c. The point of the code is for now being able to actually **have the estimation of the Q-function on the next states (as required by bellman)** under the NE condition. This translates into the need of using the **old q-function on the new states to get an estimate of how the agent expects to get in as a value if both were playing optimally.**

> q bar is like the expected value of q on these new states, which is mathematically translated into sum + prob weight (i.e. scalar prod of vectors). The bar means estimate

d. Summarizing, we use the old q-function on new states to get the policy there. By multiplying this policies (vectors) with the old q-function on new states, we get a scalar estimate of the value (discounted cumulative reward). [*]

[*]
We need to get a value estimation, so we perform it...

$$\tilde{q}_i(s, v_e, v_a, \pi^*_e, \pi^*_a) = \tilde{v}_i(s, v_e, v_a, \pi^*_e, \pi^*_a) = \sum_{a_e \in A_e} \sum_{a_a \in A_a} \pi^*_e(a_e|s, \dots)\pi^*_a(a_a|s, \dots) q^*_i(s, v_e, v_a, a_e, a_a)$$

Why so?

> In general we have that
>
> $$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s] = \underbrace{\sum_{a \in A} \Big(\pi(a \mid s)}_{\substack{\text{each action has its probability} \\ \text{of being chosen from state s}}} \cdot \Big(\underbrace{\sum_{r \in R} \sum_{s' \in S} p(s', r \mid s, a)}_{\substack{\text{each action leads to} \\ \text{different possible rewards} \\ \text{and reached states... having} \\ \text{their probability to occur} \\ \text{after acting with a from state s}}} \cdot \Big(\underbrace{r}_{\text{immediate reward}} + \underbrace{\gamma\, v_\pi(s')}_{\substack{\text{discounted state-value} \\ \text{function from next state}}} \Big)\Big)\Big)$$
>
> And we know we can express with q:
>
> $$q_\pi(s, a) = \sum_{s' \in S} \sum_{r \in R} p(s', r \mid s, a) \Big(r + \gamma\, v_\pi(s') \Big)$$
>
> Which is the inner parentesis of the previous one. So:
>
> $$v_\pi(s) = \sum_{a \in A} \pi(a \mid s)\, q_\pi(s, a)$$
>
> Which directly reduces to the initial one.